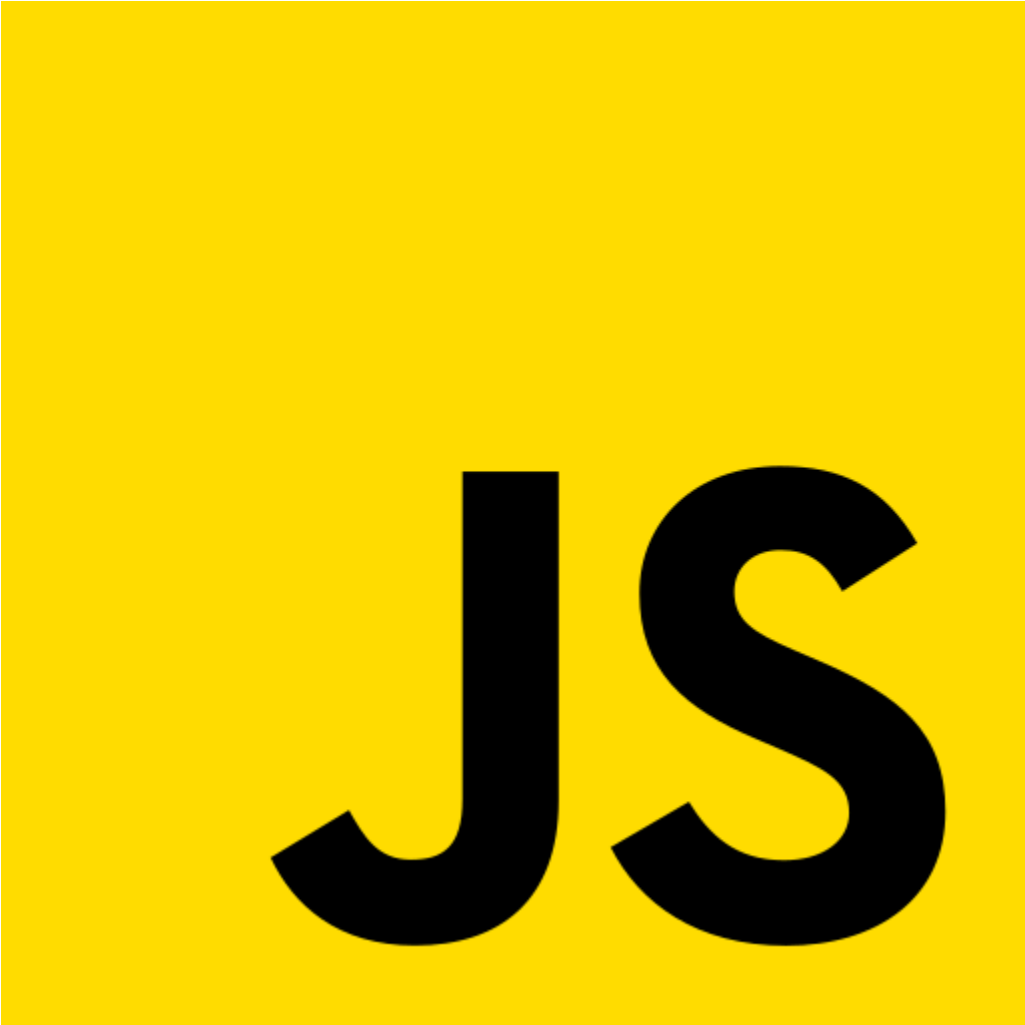


JavaScript

The logo consists of a solid yellow square. In the center of the square, the letters 'JS' are written in a large, bold, black, sans-serif font.

JS

Introduction to JavaScript

What is JavaScript?

JavaScript is a high-level, dynamic, and interpreted programming language primarily used for client-side scripting on the web. It's executed on the client side (in the user's web browser) rather than on the server side, allowing for interactive web pages and applications.

JavaScript used for:

- Creating responsive and interactive web pages
- Developing desktop and mobile applications
- Building server-side applications with technologies like Node.js
- Creating games and animations
- Adding functionality to web pages, such as validations and calculations

History

JavaScript was created by **Brendan Eich** in 1995 while working at Netscape Communications Corporation. Originally named Mocha, then LiveScript, and finally JavaScript, it was designed in just 10 days. In 1996, JavaScript was submitted to ECMA International for standardization, leading to ECMAScript as its standardized specification. ECMAScript continued evolving, with ECMAScript 5 (2009) introducing significant enhancements. ECMAScript 6 (ES6) in 2015 marked a major milestone, bringing modern features. Today, JavaScript is used in the web, server-side (Node.js), mobile apps (React Native), and desktop apps (Electron), with a thriving ecosystem of frameworks and libraries.

What is ECMA?

ECMA International, known as the European Computer Manufacturers Association, is a standards organization responsible for developing and

maintaining standards for information and communication technology (ICT) systems. They create standards for various technologies, including programming languages like JavaScript (through the ECMAScript standard), file formats, communication protocols, and more.

Keyword:

- Reserved words with special meaning in JavaScript
- Used for syntax, control flow, and declarations

Examples: if, else, for, let, const, function

Words can be used as variable names, but keywords cannot be used for variable or function names.

Compiler:

1. Translate the source code into machine code beforehand.
2. Creates an executable file.
3. Runs on the target machine without needing the compiler.
4. Errors are detected during compilation.
5. Faster execution.

Example: C++, Java (compiled to bytecode)

Interpreter:

1. Translates and executes source code line-by-line.
2. No executable file is created.
3. Requires the interpreter to run on the target machine.
4. Errors are detected during execution.

5. Slower execution.

Example: Python, JavaScript (in web browsers), Ruby

Key differences:

1. Compilation step: Compiler (before execution) vs. Interpreter (during execution)
2. Executable file: Compiler (yes) vs. Interpreter (no)
3. Error detection: Compiler (before execution) vs. Interpreter (during execution)
4. Execution speed: Compiler (faster) vs. Interpreter (slower)

Variables

A variable is a logical named container used to store data values in a computer program. It acts as a placeholder for various types of information, such as numbers, strings, Booleans, arrays, objects, and functions.

JavaScript supports three ways to declare variables: **var**, **let**, and **const**.

1. **var** was the original way to declare variables in JavaScript, but it's now less commonly used due to scoping issues (it has function scope, not block scope). They can be redeclared and reassigned within their scope.

```
var name = "John";  
  
console.log(name); // Output: John
```

2. Let: Introduced in ECMAScript 6 (ES6), variables declared with the **let** keyword are block-scoped, meaning they are only accessible within the block in which they are defined (enclosed by `{}`). They cannot be redeclared within the same scope but can be reassigned.

```
let age = 20;

console.log(age); // Output: 20

age = 21; // Reassign the value

let age = 22; // Error age cannot be redeclared
```

3. Const: Constants are declared using the **const** keyword. They are also block-scoped and cannot be reassigned after initialization. However, if the constant holds an object or array, the properties or elements of the object/array can be modified.

```
const name = "Ali";

console.log(name); // Output: Ali
```

Naming Rules:

- Variable names must begin with a letter, dollar sign (\$), or underscore (_).
- Subsequent characters can be letters, digits, dollar signs, or underscores.
- Variable names are case-sensitive.
- Avoid using reserved keywords as variable names.

Data types

A data type is a classification that specifies which type of value a variable can hold in a computer program. It defines the characteristics of the data and how it can be manipulated.

In JavaScript, data types are categorized into primitive data types and non-primitive/reference data types:

Primitive Data Types: These basic data types represent a single value.

Number: Represents numeric values like all numbers and floating-point numbers. **Example:** 10, 3.14, -5, 78

String: Represents textual data enclosed in single or double quotes. **Example:** "Hello", 'JavaScript', `C`, "This is a string", "78",

Boolean: Represents a logical value of **true** or **false**.

Undefined: Represents a variable that has been declared but not assigned a value.

Example: `var a; // type is undefined because the value is not assigned`

Null: Represents the intentional absence of any value.

Example: `var a = null;`

Non-Primitive Data Types: These data types can hold multiple values or other data types in a single variable.

Object: Represents a collection of key-value pairs where keys are strings and values can be any data type, including other objects.

Example: `const boy = {name: "Ali", age: 20}`

Array: Represents an ordered collection of elements, which can be of any data type.

Example: `let fruits = ["apple", "banana", "cherry"];`

By use of **typeof** operator data type can be checked for any variable.

```
let a = 20;
```

ie. `console.log(typeof a)` // output data type of variable a

Operators

In JavaScript, operators are symbols or keywords that perform operations on operands (variables or values). Short operators, also known as shorthand operators or compound assignment operators, are a concise way of combining arithmetic, bitwise, logical, and other operations with the assignment.

1. Arithmetic Operators (+, -, *, /, %)

Arithmetic operators are used to perform basic mathematical operations.

- +:** Adds two operands.
- :** Subtracts the right operand from the left operand.
- *:** Multiplies two operands.
- /:** Divides the left operand by the right operand.
- ?:** Returns the remainder of the division of the left operand by the right operand.

```
let a = 10;
let b = 3;
console.log(a + b); // Output: 13
console.log(a - b); // Output: 7
console.log(a * b); // Output: 30
console.log(a / b); // Output: 3.33
console.log(a % b); // Output: 1
```

2. Assignment Operators (=, +=, -=, *=, /=, %=)

Assignment operators assign values to variables.

- =** Assigns a value to a variable.
- +=** Adds the right operand to the left operand and assigns the result to the left operand.

-= Subtracts the right operand from the left operand and assigns the result to the left operand.

***=** Multiplies the left operand by the right operand and assigns the result to the left operand.

/= Divides the left operand by the right operand and assigns the result to the left operand.

%= Computes the modulus of the left operand with the right operand and assigns the result to the left operand.

```
let x = 10;
x += 5; // x = x + 5
console.log(x); // Output: 15

x -= 5; // x = x - 5
console.log(x); // Output: 10

x *= 5; // x = x * 5
console.log(x); // Output: 50

x /= 5; // x = x / 5
console.log(x); // Output: 10

x %= 5; // x = x % 5
console.log(x); // Output: 0
```

3. Comparison Operators (==, ===, !=, !==, >, <, >=, <=)

Comparison operators compare two values and return a Boolean (**true** or **false**).

== Equality - Checks if two operands are equal in value (with type conversion).

```
5 == "5" // true
```

=== Strict Equality - Checks if two operands are equal in value and data type.

```
5 === "5" // false
```

!= Inequality - Checks if two operands are not equal in value (with type conversion).

```
5 != "5" // false
```

!== Strict Inequality - Checks if two operands are not equal in value or type.

```
5 !== "5" // true
```

> Greater Than - Checks if the left operand is greater than the right operand.

```
5 < 10 // true
```

< Less Than - Checks if the left operand is less than the right operand.

```
5 > 10 // false
```

>= Greater Than or Equal To - Checks if the left operand is greater than or equal to the right operand.

```
5 >= 10 // false
```

<= Less Than or Equal To - Checks if the left operand is less than or equal to the right operand.

```
5 <= 10 // true
```

4. Logical Operators (&&, ||, !)

Logical operators are used to combine conditional statements.

&& Logical AND - Returns true if both operands are true.

|| Logical OR - Returns true if at least one operand is true.

! Logical NOT - Returns true if the operand is false and vice versa

```
true && false // false
```

```
true || false // true
!true // false
```

5. Unary Operators(Increment/Decrement ++ , --)

Operators with one operand.

- **++i or i++:** Increment - Adds 1 to the operand.
- **--i or i--:** Decrement - Subtracts 1 from the operand.

```
let a = 5;
console.log(++a); // Pre-increment: Output: 6
console.log(a++); // Post-increment: Output: 6
```

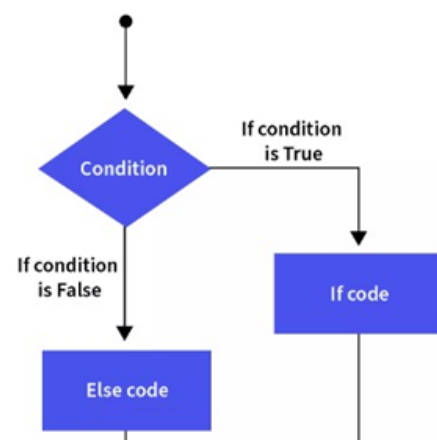
Conditional Statements (if – else)

Conditional statements in JavaScript, such as **if**, **else if**, and **else**, are used to make decisions in code based on certain conditions. They allow you to execute different blocks of code depending on whether a condition evaluates to true or false.

If-Else: Executes code based on conditions. When the condition is true execute if block if the condition is false then else block execute.

```
let age = 20;

if (age >= 18) {
  console.log("You are an adult.");
}
```



```
} else {  
    console.log("You are a minor.");  
}  
  
// Output: You are an adult.
```

else if statement: It allows you to check multiple conditions. If the first **if** condition is false, it checks the next **else if** condition, and so on.

```
let score = 75;  
  
if (score >= 90) {  
    console.log("Grade: A");  
} else if (score >= 80) {  
    console.log("Grade: B");  
} else if (score >= 70) {  
    console.log("Grade: C");  
} else {  
    console.log("Grade: F");  
}  
  
// Output: Grade: C
```

Loops

A loop is a programming construct that allows a block of code to be repeated a certain number of times, or until a specific condition is met. Loops are used to perform repetitive tasks, iterate over data, and control the flow of a program.

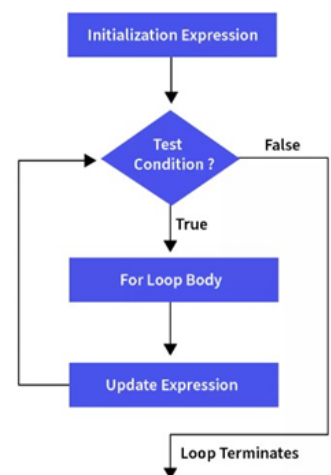
Loops are useful for:

- Repeating a task multiple times
- Iterating over data structures such as arrays or objects
- Implementing algorithms that require repetition
- Controlling the flow of a program

For Loop

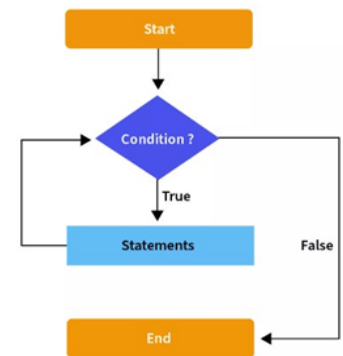
The **for** loop is used when the number of iterations is known before end. It consists of three parts: initialization, condition, and increment/decrement.

```
for (let i = 1; i <= 3; i++) {  
    console.log(`Iteration ${i}`);  
}  
// Output:  
// Iteration 1, Iteration 2, Iteration 3
```



While Loop

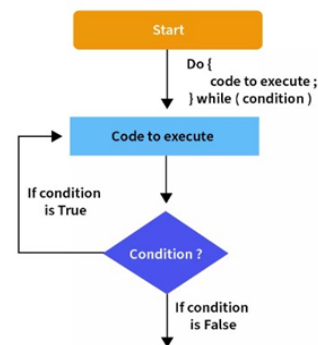
The **while** loop executes a block of code as long as the condition evaluates to **true**. Unlike a for loop where you specify the number of iterations, a while loop relies solely on the condition to control the loop's execution.



```
let count = 1;
while (count <= 3) {
  console.log(`Count: ${count}`);
  count++;
}
// Output: Count: 1, Count: 2, Count: 3
```

Do While Loop

The **do...while** loop is similar to the **while** loop, but it ensures that the block of code is executed at least once, even if the condition is **false**.



```
let num = 1;
do {
  console.log(`Number: ${num}`);
  num++;
} while (num >= 5);
```

```
// Output:  
// Number: 1 because the condition is false
```

Break Statement

The **break** statement is used to terminate the execution of a loop (for, while, do...while) or a switch statement prematurely. When encountered, the **break** statement immediately exits the loop or switch block and continues with the next statement after the loop or switch.

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) {  
    break // Exit the loop when i is 3  }  
  }  
  console.log(i);  
} // Output: 0, 1, 2
```

Continue Statement:

The **continue** statement is used to skip the current iteration of a loop and continue with the next iteration. When encountered, the **continue** statement skips the remaining code in the loop for the current iteration and proceeds with the next iteration.


```
for (let i = 0; i < 5; i++) {  
  if (i === 2) {  
    continue    // Skip the iteration when i is 2  }  
  }  
  
  console.log(i);  
}  
  
// Output: 0, 1, 3, 4
```

Functions

Functions are reusable blocks of code designed to perform a specific task.

It helps to:

- Organize code
- Reduce repetition
- Make code reusable
- Simplify complex tasks

```
//=== Function Declaration ===//

function greet(name) {
    return `Hello, ${name}!`;
}

//= Call the function =//

const result = greet("Ali");
console.log(result); // "Hello, Ali!"

//=== Default Parameters ===//

function sayHi(name = "Guest") {
    return `Hi, ${name}`;
}

console.log(sayHi()); // "Hi, Guest"

//=== Arrow Function(ES6)===//

const multiply = (x, y) => {
    return x * y;
}

//===== Arrow Function Short Syntax

const multiply = (x, y) => x * y;
```

String Methods

String methods are functions that operate on strings in JavaScript. They allow you to manipulate and work with text data efficiently. Some common string methods include:

length: Returns the length of a string, i.e., the number of characters it contains.

```
const str = "Hello, world!";  
console.log(str.length); // 13
```

concat(string): Combines two or more strings and returns a new string.

```
const str1 = "Hello ";  
const str2 = "World";  
const str3 = str1.concat(str2);  
console.log(str3); // Hello World
```

toUpperCase(): Converts all characters in a string to uppercase.

toLowerCase(): Converts all characters in a string to lowercase.

```
let str = "Hello, World!";  
console.log(str.toUpperCase()); // "HELLO, WORLD!"  
console.log(str.toLowerCase()); // "hello, world!"
```

trim(): Removes whitespace from both ends of a string.

```
const str = "  Hello!  ";  
console.log(str.trim()); // "Hello!"
```

replace(find, replace): Replaces occurrences of a specified substring with another string.

```
let str = "Hello, world!";  
console.log(str.replace("world", "JavaScript"));  
// "Hello, JavaScript!"
```

Array and its Methods

An **array** is a collection of elements stored in a single variable. Each element in the array is identified by its index number, starting from 0. Arrays can hold multiple values of the same or different data types (like numbers, strings, objects, etc.).

```
let fruits = ["apple", "banana", "cherry"];  
console.log(fruits);  
//Output: ["apple", "banana", "cherry"]  
console.log(fruits[0]); // Output: "apple"  
console.log(fruits[1]); // Output: "banana"
```

length: Returns the number of elements in an array.

```
const arr = [1, 2, 3];  
console.log(arr.length); // 3
```

push(): Adds one or more elements to the end of an array and returns the new length of the array.

```
const arr = [1, 2, 3];  
arr.push(4);  
console.log(arr); // [1, 2, 3, 4]
```

pop(): Removes the last element from an array and returns that element.

```
const arr = [1, 2, 3, 4];  
console.log(arr.pop()); // 4
```

shift(): Removes the first element from an array and returns that element, shifting all other elements down by one index.

```
const arr = [1, 2, 3, 4];  
console.log(arr.shift()); // 1
```

unshift(): Adds one or more elements to the beginning of an array and returns the new length of the array.

```
let arr = [ 2, 3];  
arr.unshift(0);  
console.log(arr); // [0, 2, 3]
```

reverse(): Reverses the order of elements in an array.

```
let arr = [1, 2, 3, 4];  
arr.reverse();  
Console.log(arr); // [4, 3, 2, 1]
```

concat(array): Combines two or more arrays and returns a new array.

```
let arr = [1, 2, 3, 4];  
const arr2 = [4, 5];  
console.log(arr.concat(arr2)); // [1, 2, 3, 4, 4, 5]
```

Objects

In JavaScript, objects are a fundamental data type representing a collection of key-value pairs. They are used to store and manipulate data in a structured way.

```
const person = {  
  name: "John",  
  age: 30,  
  city: "Lahore"  
};  
console.log( person.name ); // "John"  
console.log( person['name'] );  
  
//Add, Update, Delete Object Properties  
person.address = "New York"; // Add  
person.age = 31;           // Update  
delete person.city;        // Delete
```

JavaScript **for...in** vs **for...of** Loops

◆ 1. for...in Loop

- Used to **iterate over the keys (property names)** of an **object**.
- Can also be used on arrays, but it gives **indexes (keys)**, not values (so not recommended for arrays).

```
const student = { name: "Ali", age: 21, grade: "A" };

for (let key in student) {
  console.log(key, ":", student[key]);
}
// name : Ali
// age : 21
// grade : A
```

```
const fruits = ["Apple", "Banana", "Mango"];

for (let index in fruits) {
  console.log(index, ":", fruits[index]);
}
// 0 : Apple
// 1 : Banana
// 2 : Mango
```

👉 Use `for...in` when you need keys/properties.

◆ 2. for...of Loop

- Used to **iterate over values** of an **iterable** (like arrays, strings, maps, sets).
- **Does not work on objects directly** (unless you use `Object.keys`, `Object.values`, or `Object.entries`).

```
const fruits = ["Apple", "Banana", "Mango"];

for (let fruit of fruits) {
  console.log(fruit);
}
// Apple
// Banana
// Mango
```

📌 Key Differences Table

Feature	for...in	for...of
Works on	Objects, Arrays (keys)	Iterables (Arrays, Strings, Sets, Maps)
Returns	Keys / property names	Values
Best use case	Loop through object properties	Loop through array values

2. Callback Function

- A **callback function** is a function that is **passed as an argument** to another function.
- It allows asynchronous or repeated execution.

```
function greet(name, callback) {  
  console.log("Hello " + name);  
  callback();  
}  
  
function sayBye() {  
  console.log("Goodbye!");  
}  
  
greet("Ali", sayBye);  
// Hello Ali  
// Goodbye!
```

👉 Here, `sayBye` is passed as a **callback** to `greet`.

◆ 1. forEach Loop

- **forEach()** is a built-in array method in JavaScript.
- It executes a callback function once for each element in the array.
- Does not return a new array (unlike **map**).

```
const fruits = ["Apple", "Banana", "Mango"];

fruits.forEach(function(fruit, index) {
  console.log(index, ":", fruit);
});
// 0 : Apple
// 1 : Banana
// 2 : Mango
```

👉 Here, `function(fruit, index)` is the **callback function**.

DOM (Document Object Model)

The Document Object Model (DOM) in JavaScript is a programming interface that represents the structure of a web document as a tree of objects. Each element, attribute, and text node in an HTML or XML document is defined as a node in the DOM tree. JavaScript interacts with the DOM to dynamically manipulate the content, structure, and style of a web page.

The DOM allows JavaScript to:

- Access and manipulate elements and their properties
- Add, remove, and modify elements and their content
- Handle events and user interactions
- Traverse and search the document tree

```
<html>

  <body>

    <h1>Title</h1>

    <p>Paragraph <span>hello</span></p>

  </body>

</html>

// The document in tree view

Document
├─ html
│   └─ body
│       ├── h1
│       └─ p
```

1. Accessing DOM Elements

Access elements to read or manipulate them.

Example 1: `document.getElementById`

```
<div id="example">Hello, World!</div>

<script>

  const element = document.getElementById("example");

  console.log(element.textContent);

  // Output: "Hello,   World!"

</script>
```

Example 2: `document.querySelector(".text")` or `("#id")`

```
<p class="text">This is a paragraph.</p>

<script>

  const para = document.querySelector(".text");

  console.log(para.textContent);

  // Output: "This is a paragraph."

</script>
```

By Class: `document.getElementsByClassName("class")`

```
<div class="example">Hello, World!</div>

<script>

// Get the element

let element = document.getElementsByClassName("example")[0];

// Get the text content

let text = element.textContent;

// Set the text content

element.textContent = "Hello, Universe!";

</script>
```

All Matches: `document.querySelectorAll("selector")`

Select all the match elements (with id or class) and make an array of these elements.

```
<p class="example">Paragraph 1</p>
<p class="example">Paragraph 2</p>
<p class="example">Paragraph 3</p>
<script>
  const paragraphs = document.querySelectorAll(".example");
  paragraphs.forEach((p, index) => {
    p.textContent = `Updated Paragraph ${index + 1}`;
  });
</script>
```

Modifying Elements

```
const heading = document.getElementById("main-title");
heading.textContent = "New Title";
heading.style.color = "blue";
```

`document.getElementById("main-title")`

- This function accesses the **DOM tree** and finds the element with the ID **main-title**.

- It returns a **reference to that element node**, which you can then manipulate.

```
heading.textContent = "New Title";
```

- This changes the **text inside the HTML element** to "New Title".
- It only changes the **text**, not the HTML structure or style.

```
heading.style.color = "blue";
```

- This modifies the **inline CSS style** of the selected element.
- You are accessing the **style** property (an object) and changing its **color** property to "blue".

Creating & Inserting Elements

```
const newDiv = document.createElement("div");  
  
newDiv.textContent = "I am new here!";  
  
document.body.appendChild(newDiv);
```

```
document.createElement("div")
```

- This **creates a new element node** (`<div>`) in memory but doesn't add it to the page yet.
- It's like preparing a new HTML tag to be placed somewhere.

```
newDiv.textContent = "I am new here!";
```

- This sets the text inside the new `<div>` element.

```
y.appendChild(newDiv);
```

- This **inserts the new element** as the **last child of the** `<body>` tag.
- The `appendChild()` method physically attaches the node to the document.

Removing Elements

```
const paragraph = document.querySelector("p");  
paragraph.remove();
```

```
document.querySelector("p")
```

- Selects the **first** `<p>` **element** in the document.

- `querySelector()` is very flexible and accepts any CSS selector.

`paragraph.remove();`

- This **removes the selected element** from the DOM entirely.
- The element no longer exists on the page after this line runs.

Action	Method Used	Description
Modify content	<code>textContent</code> , <code>style</code>	Changes inner text and CSS
Create element	<code>createElement()</code>	Creates a new element in memory
Insert element	<code>appendChild()</code>	Adds a node to the DOM
Remove element	<code>remove()</code>	Deletes a node from the DOM

Events

Events are actions or occurrences that happen in the browser, such as when a user clicks a button, moves the mouse, types in a text field, or loads a page. JavaScript provides the ability to handle these events and perform specific actions when they occur.

1. Mouse Events:

click – Triggered when an element is clicked.

dblclick – Triggered on a double-click.

mousedown / mouseup – Triggered when a mouse button is pressed/released.

mouseover / mouseout – Triggered when the mouse enters or leaves an element.

```
<div id="box" style="width: 50px; height: 50px; background: red;"></div>

<script>
  const box = document.getElementById("box");
  box.addEventListener("mouseover", () => {
    box.style.backgroundColor = "blue";
  });

  box.addEventListener("mouseout", () => {
    box.style.backgroundColor = "red";
  });
</script>
```

2. Keyboard Events:

keydown – Triggered when a key is pressed.

keypress – Triggered when a key is pressed and held (deprecated).

keyup – Triggered when a key is released.

```
<input type="text" id="input" placeholder="type...">

<script>

const input = document.getElementById("input");

input.addEventListener("keydown", (event) => {

  console.log(`Key pressed: ${event.key}`);

});

input.addEventListener("keyup", () => {

  console.log(`Input value: ${input.value}`);

});

</script>
```

3. Form Events:

submit – Triggered when a form is submitted.

change – Triggered when the value of an input, select, or textarea changes.

```
<form id="myForm">

  <input type="text" placeholder="Enter..." required>

  <button type="submit">Submit</button>

</form>

<script>

  const form = document.getElementById("myForm");

  form.addEventListener("submit", (event) => {

    event.preventDefault(); // Prevent to submit

    alert("Form submitted!");

  });

</script>
```